

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

SE 423: Introduction to Mechatronics

Lecture 19: Trajectory planning

Marius Juston

Wednesday, April 1st, 2026

Review – 5-minute partner discussion

Talk to the person next to you and answer these questions.

- If you had multi-goals that you were trying to reach what would use A*, Dijkstra, Greedy Best First, or BFS? What about a single goal?
- In the previous lecture we performed path planning on a grid graph, what did each vertex in the grid represent (i.e. what was the state space that A* was searching through?)
- What kind robot is most closely represented when you perform path planning with the state only being (x, y)?
- For a differential robot, is the movement cost uniform based purely on the position on the robot? The movement cost from $(x, y) \rightarrow (x + 1, y) = \text{or} \neq (x, y) \rightarrow (x, y + 1)$
- What is the state space of differential robot?

Reminders

Office Hours:

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

Lab: Starting Lab 6 this week. This a 3-week lab!

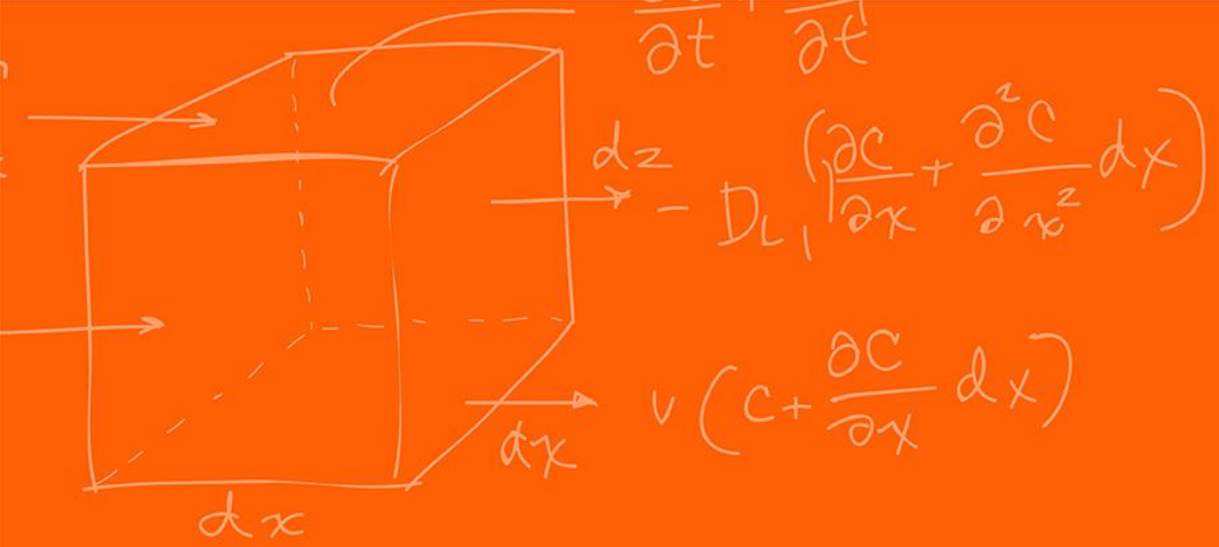
- Don’t forget to have the Lab LabVIEW application implemented before lab.

Homeworks:

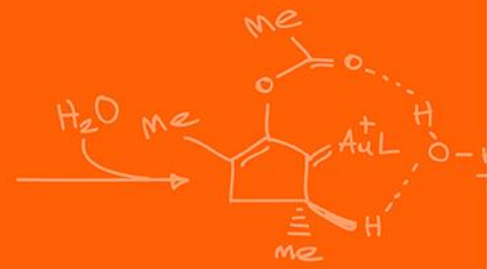
- Check-offs for homework 4 Ex 4 are due **April 7, 5PM (The end of office hours)**
- Remainder of homework 4 due on Gradescope on **April 8, 9AM**
- Next homework is primarily the **personal project** due on **April 22, 9AM**.
  - I would highly recommend you thinking now and maybe even starting it now if you have a good idea already!

Questions?

Homework, Lab, LabVIEW?

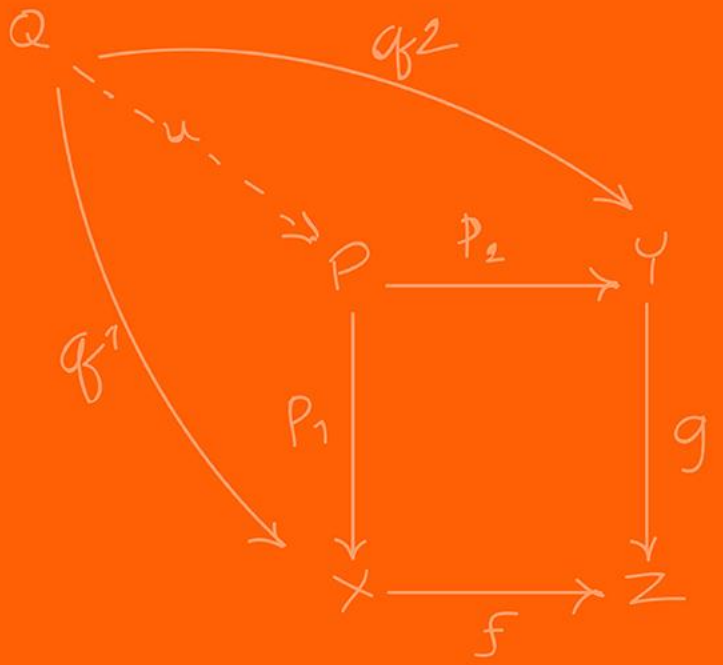








Path planning

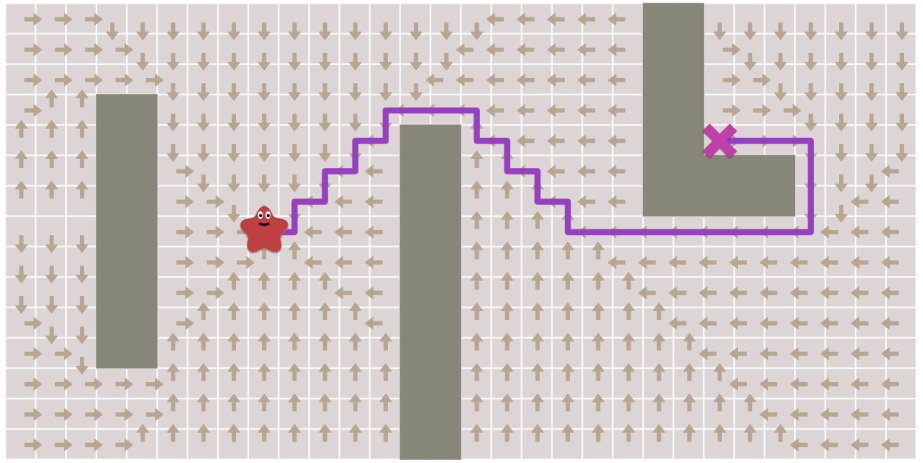


Path Planning

In the previous lecture we explored A* on a 2D grid, where we represented the environment as a graph discrete nodes.

What was the output path returned?

The output path was a sequence of (x, y) waypoints.



In lecture 15, we however, define the robot's pose $p = (x, y, \theta)$ in the global frame which moves using the linear velocity V and angular ω.

Path Planning

Standard A* assumes the robot is a "point mass" that can move in any direction!

However, differential drive robots cannot translate sideways! A drone, however, would be fine with this “point mass” definition.

When the path goes from moving "North" to "East", the angle changes by 90 degrees instantly.

This implies an infinite angular velocity at that corner if you are not already facing that direction. What happens to our DC motors if we command an infinite angular velocity?
How can we handle this?

Stop and turn or try tracking the next waypoint.

Path Planning

One common approach is to ignore the robot's kinematics during planning.

1. Plan in 2D (x, y) space using A*.
2. Build a continuous feedback controller to "chase" the discrete path.
3. We infer the desired heading θ_r from the angle between waypoints:

$$\theta_r = \text{atan2}(\Delta y, \Delta x)$$

Path Planning

- One possible way is to define the error state $\mathbf{e} = [e_x, e_y, e_\theta]^T$ transformed into the robot's local coordinate frame:

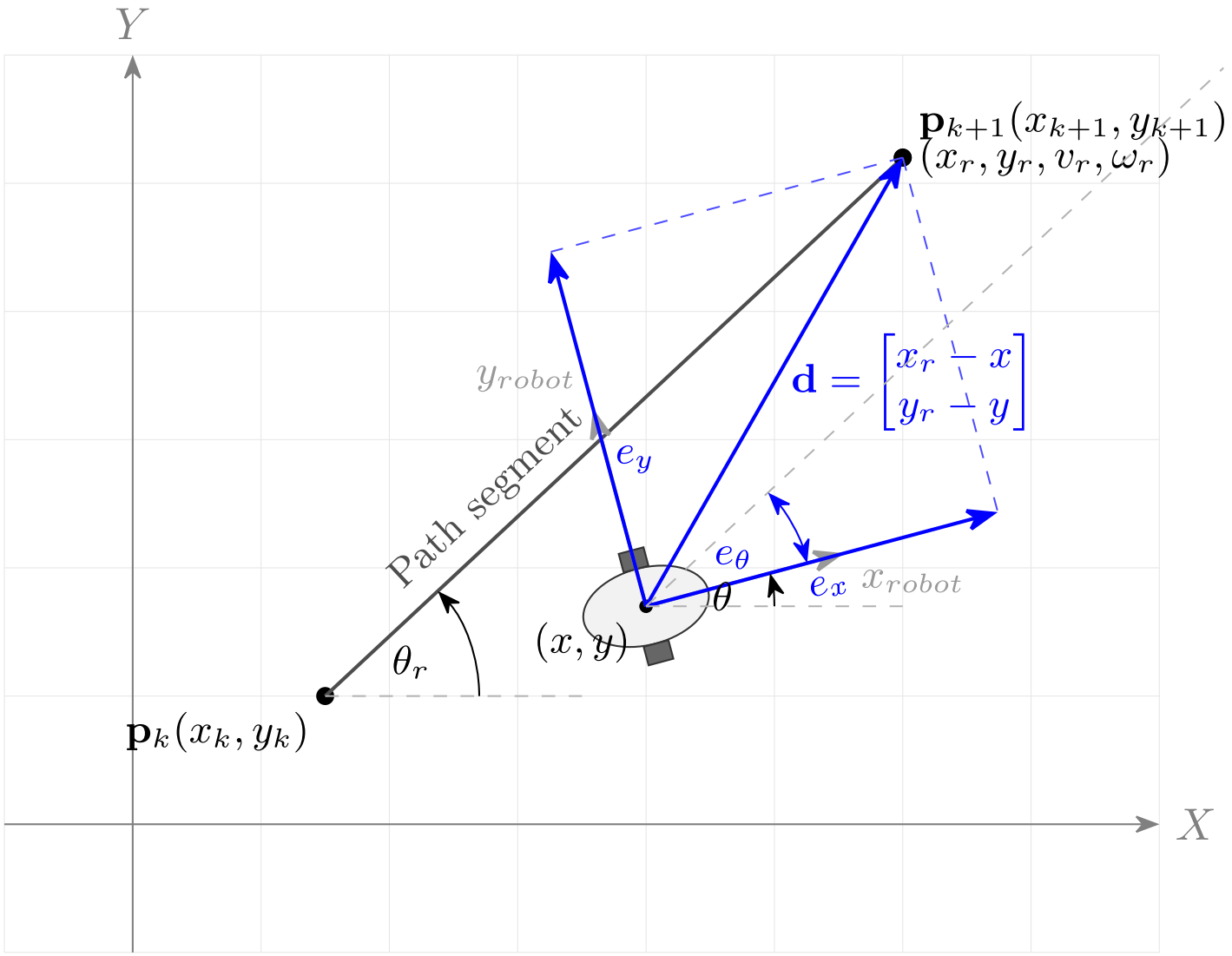
$$\begin{bmatrix} e_x \\ e_y \\ e_\theta \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x \\ y_r - y \\ \theta_r - \theta \end{bmatrix}$$

Where (x_r, y_r, θ_r), represents the waypoint coordinate from A*.

The kinematic model governing the motion of a unicycle (which accurately represents the center point of a differential drive robot) is:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Path Planning



Taking the time derivative of this error vector and substituting our kinematic equations yields the open-loop error dynamics:

$$\dot{e_x} = \omega e_y - v + v_r \cos(e_\theta)$$
$$\dot{e_y} = -\omega e_x + v_r \sin(e_\theta)$$
$$\dot{e_\theta} = \omega_r - \omega$$

To generate control actions that guarantee the robot will converge to the path, we can utilize a non-linear feedback tracking controller (originally proposed by Kanayama et al.).
We propose the following control law using positive gain constants $K_x, K_y, K_\theta > 0$:

$$v = v_r \cos(e_\theta) + K_x e_x$$
$$\omega = \omega_r + v_r\left(K_y e_y + K_\theta \sin(e_\theta)\right)$$

Path Planning

- With non-linear control stability theory (Lyapunov), you can prove that the previously provided controller will converge to the path.

You can define a positive-definite Lyapunov candidate function $V(\mathbf{e})$:

$$V = \frac{1}{2}\left(e_x^2 + e_y^2\right) + \frac{1 - \cos(e_\theta)}{K_y}$$

We must prove that its time derivative $\dot{V}$ is negative semi-definite. Taking the derivative:

$$\dot{V} = e_x \dot{e_x} + e_y \dot{e_y} + \frac{\sin(e_\theta)}{K_y}\dot{e_\theta}$$

I will leave as an exercise to you to verify that as long as $K_x, K_y, K_\theta > 0$ and $v_r > 0$. The error states asymptotically converge to zero.

Path Planning

So, we have decoupled the control from the robot's planner. However, doing so we could have potential problems. What might they be?

When inferring θ from a jagged grid path, the reference heading θ_r is discontinuous.

This creates sudden spikes in heading error (e_θ).

The feedback controller saturates the motors trying to fix the error.

The robot swings wide around corners, potentially crashing into obstacles the planner thought were safe.

The path planner does not understand the robot's kinematics when planning!

Path Planning

Path Planning

This decoupled controller, which pursues a target point and has the controller constantly chasing after it is called **Pure Pursuit**

There are other ways of implementing the trajectory following, among which we will talk about below. Some other ways revolve around solving optimization problems.

Path Planning

- To fix this, the planner must understand the robot's physical heading.

We must expand our search state from 2D (x, y) to 3D (x, y, θ).

The robot now searches through **positions and orientations**! A state transition is no longer just moving to an adjacent grid cell.

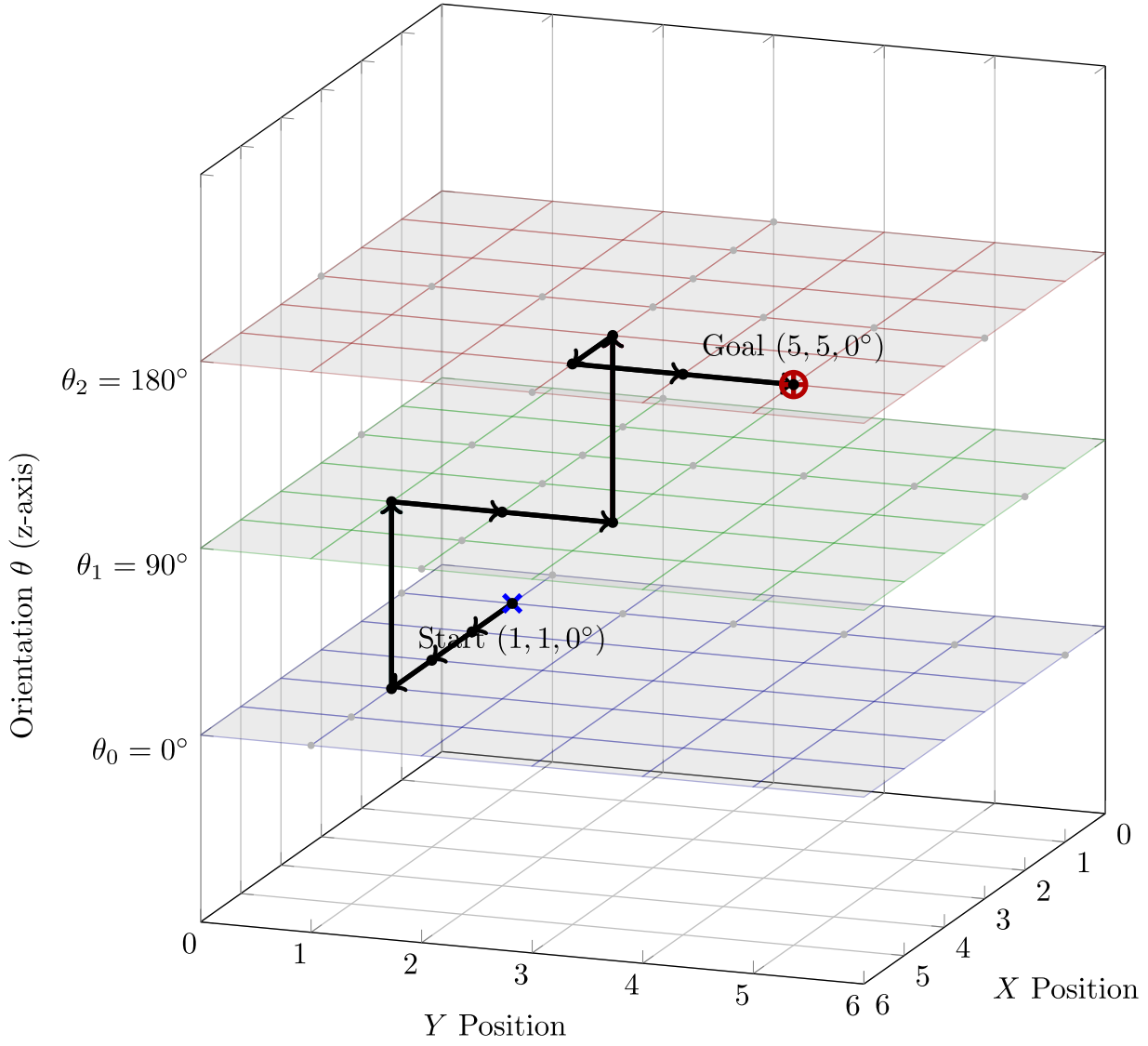
We can **discretize θ into bins** (e.g., 8 directions, 16 directions).

From a state $(x, y, 0^\circ)$, valid moves might be "drive forward at 0°" or "turn to 45° and drive".

Given that we have the state θ, we can give a **penalty for $\Delta\theta$** between states to make robots move less.

This ensures the path considers the robot's current orientation. However, true non-holonomic robots don't just "turn and drive"—they curve.

Path Planning



The Curse of Dimensionality

- Adding θ dramatically increases the search space.

If a 100x100 grid has 10,000 states in 2D discretizing θ into 72 angles (every 5 degrees) yields **720,000 states**!

How does this affect the memory and compute time required for A*?

Every time you add a dimension you increase the number of possibilities exponentially!

This phenomenon is called the **Curse of Dimensionality** and is a problem is a lot of applications of math!

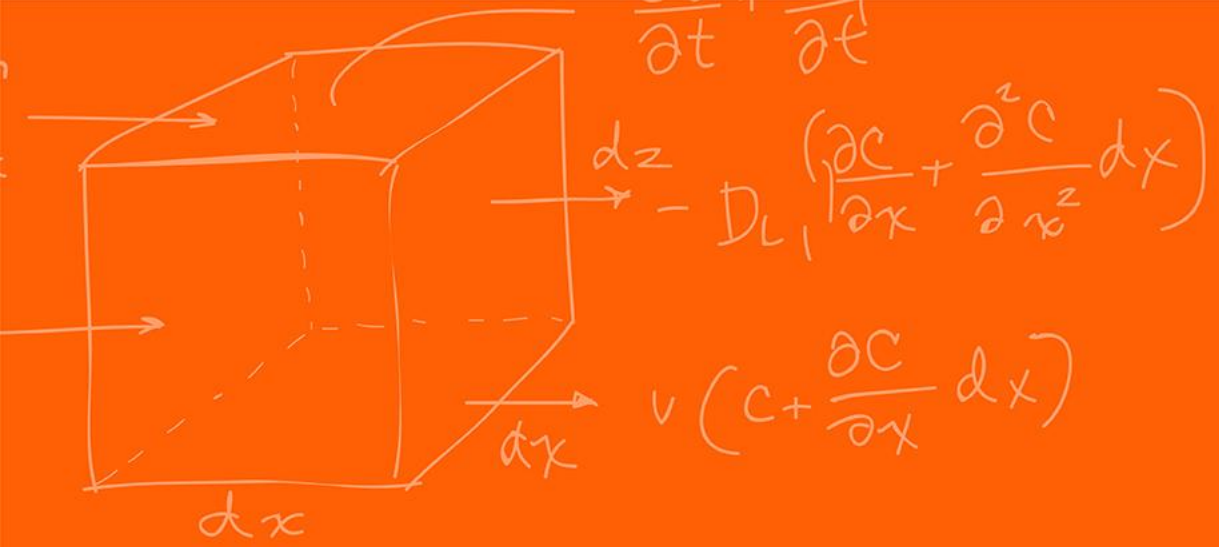
There is no way to escape this problem!

The Curse of Dimensionality

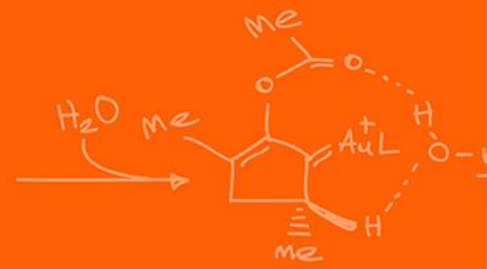
Before we commit to heavy 3D planning, let's establish when 2D is acceptable:

- 2D planning works well for omnidirectional robots (Mecanum wheels).
- It also works for very slow differential drive robots in wide-open spaces (you can have 8-connected neighbors and tune your controller to minimize edge clipping).

It fails catastrophically for cars (Ackermann steering) or robots in tight, cluttered corridors.

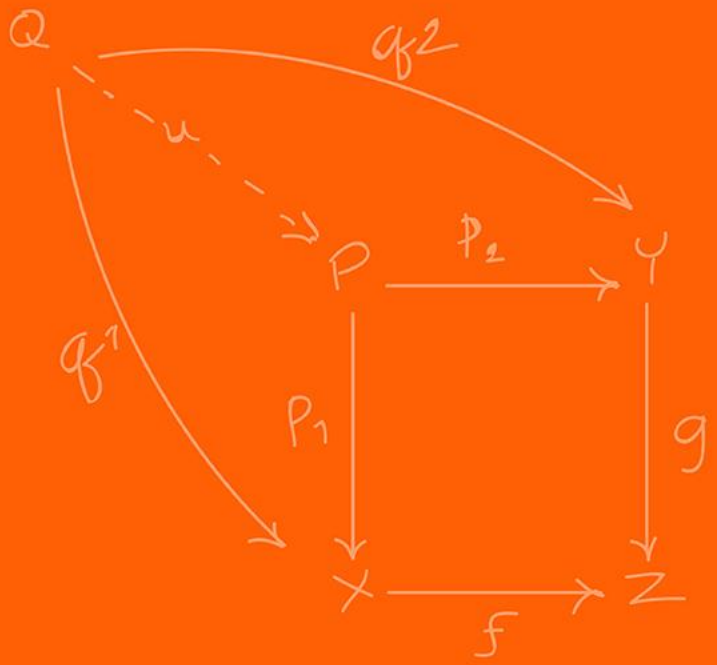








Hybrid A*



Hybrid A*

- Instead of expanding to adjacent grid cells, Hybrid A* simulates the kinematics.

It applies a set of discrete control commands (V, ω) for a short time step Δt.

The end points of these simulated curves become the new child nodes.

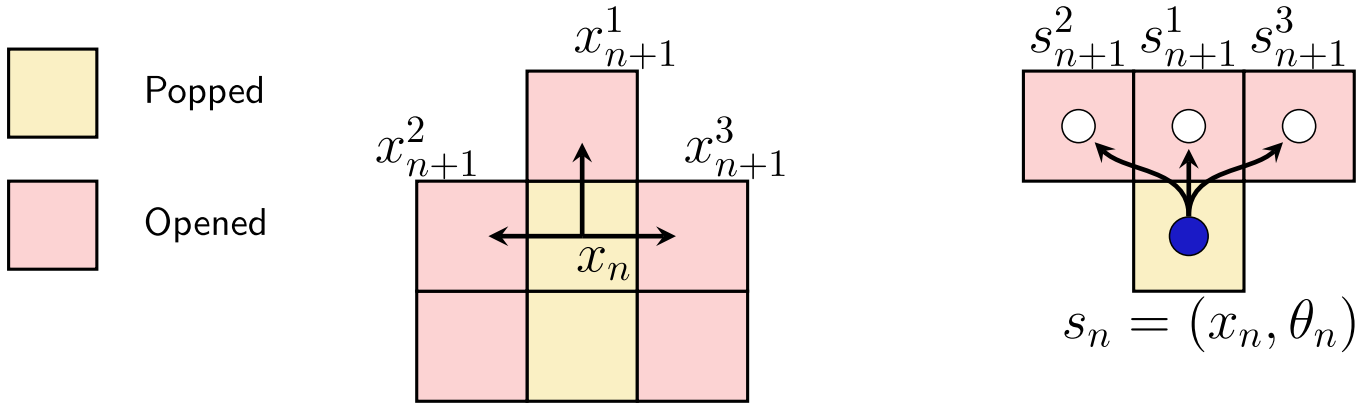
For differential drive robots, we can also command $V = 0$ and $\omega \neq 0$. This allows the robot to turn in place (a zero-point turn). Hybrid A* can include these zero-point turns as valid node expansions.

Why might we want to assign a high "cost penalty" to zero-point turns in our algorithm?

Hybrid A*

- Hybrid A* uses a discrete 3D grid (x, y, θ) purely for spatial hashing (to know if an area was visited). However, the actual coordinates stored in the nodes are continuous floating-point values.

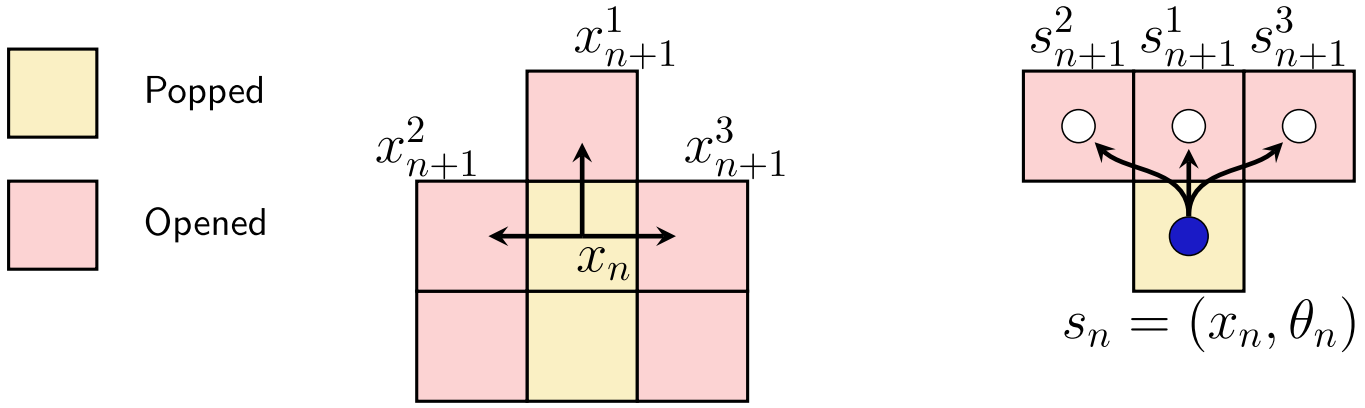
This guarantees that the generated path is physically drivable and C^1 continuous (smooth).There are no impossible 90-degree instantaneous turns!



A* Hybrid A*

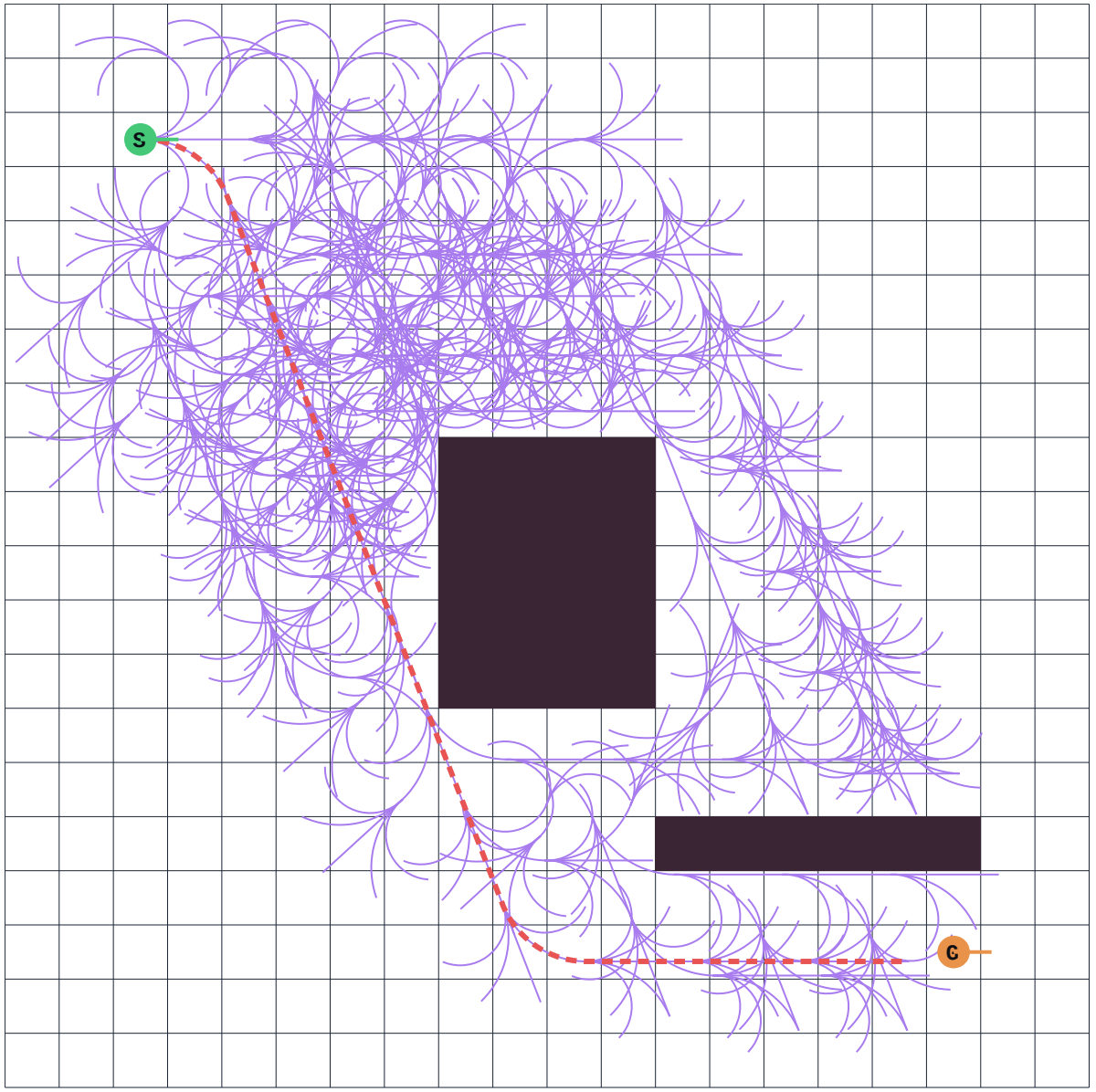
Hybrid A*

- Computing the cost of each state is relatively similar. We determine the cost by considering the length of the arcs between s_n and s_{n+1}



A* Hybrid A*

Hybrid A*



Hybrid A*

- Because expanding kinematic curves is slow, Hybrid A* needs incredibly strong heuristics.

It uses a combination of two heuristics:

- The Non-Holonomic Heuristic (ignores obstacles, respects turning radius).
- The Holonomic Heuristic (respects obstacles, ignores turning radius).

The algorithm takes the maximum of these two values.

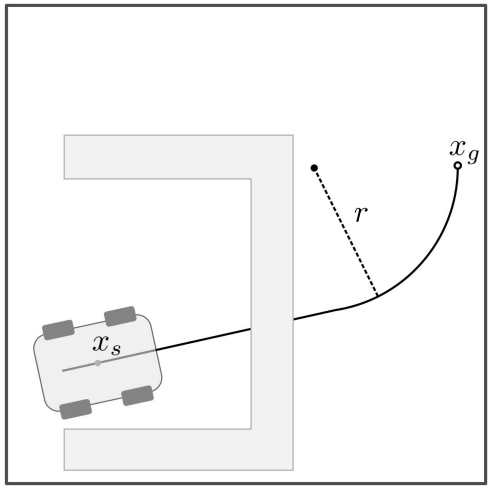
$$f(n) = g(n) + \max(h_1, h_2)$$

Hybrid A* - Holonomic Heuristic (Obstacle Aware)

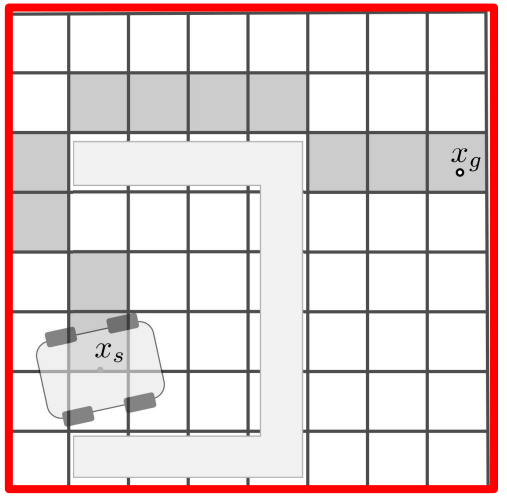
This is simply a standard 2D grid A* (or Dijkstra's) running in the background!

It calculates the distance to the goal avoiding all walls.

It prevents the kinematic search from getting stuck in dead ends. Because it's 2D, it computes very quickly.



(c) The constrained heuristic heavily underestimating the cost of the path, due to ignoring obstacles.



(d) The unconstrained heuristic accounting for obstacles and dead ends.

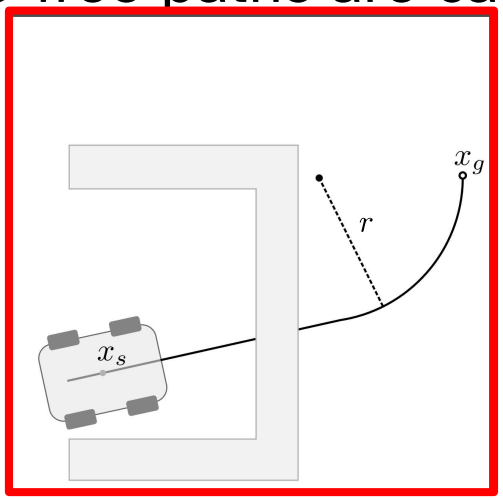
https://kth.diva-portal.org/smash/record.jsf?pid=diva2%3A1057261&dswid=4852

Hybrid A* - Non-Holonomic Heuristic (Turning Aware)

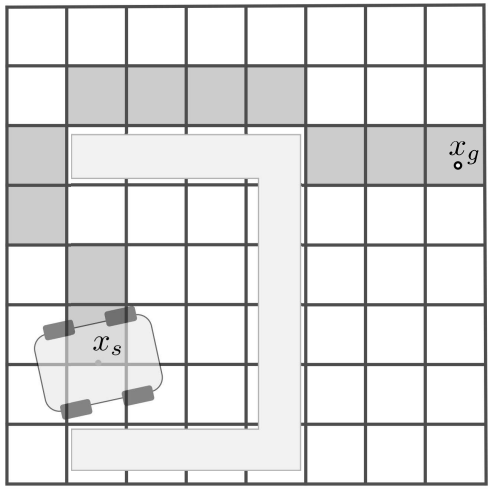
This heuristic ignores all obstacles but calculates the mathematical shortest path for a vehicle with a minimum turning radius.

If the robot needs to arrive at the goal facing a specific direction, it computes the required curves.

These optimal obstacle-free paths are called **Dubins curves** or **Reeds-Shepp** curves.

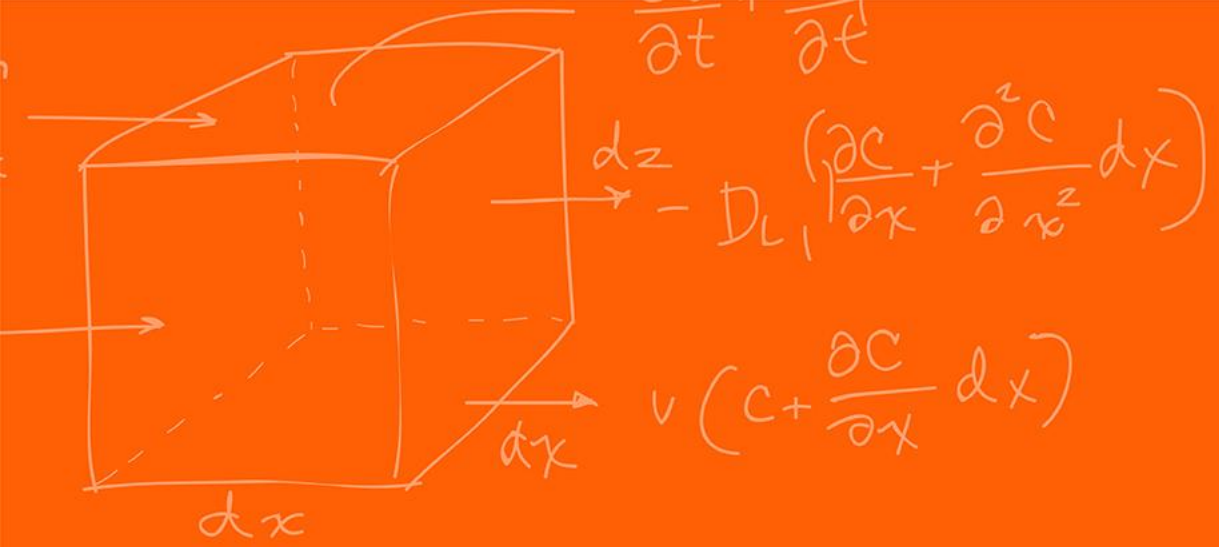


(c) The constrained heuristic heavily underestimating the cost of the path, due to ignoring obstacles.

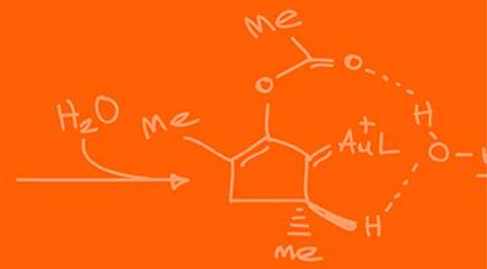


(d) The unconstrained heuristic accounting for obstacles and dead ends.

https://kth.diva-portal.org/smash/record.jsf?pid=diva2%3A1057261&dswid=4852

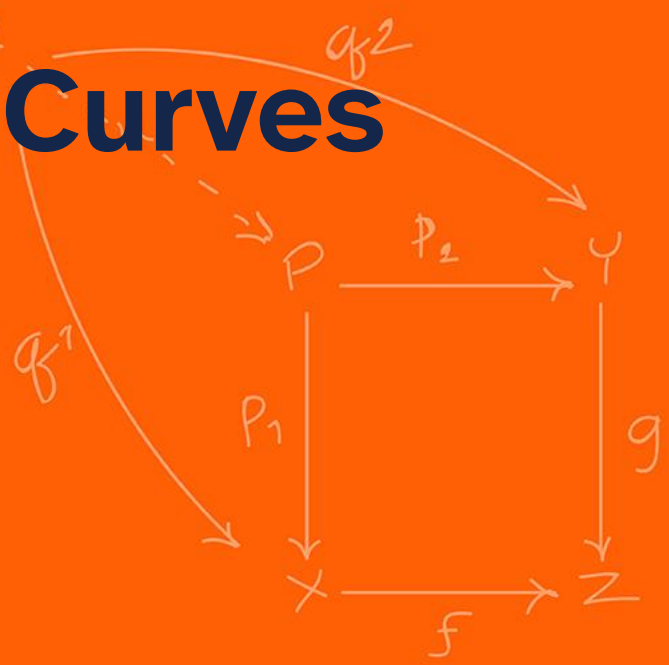








Dubins and Reeds-Shepp Curves



Hybrid A* - Dubins curves

Dubins curves find the shortest path between two (x, y, θ) states using only **forward** motion.

They consist of maximum-steering arcs (Right or Left) and Straight-line segments.

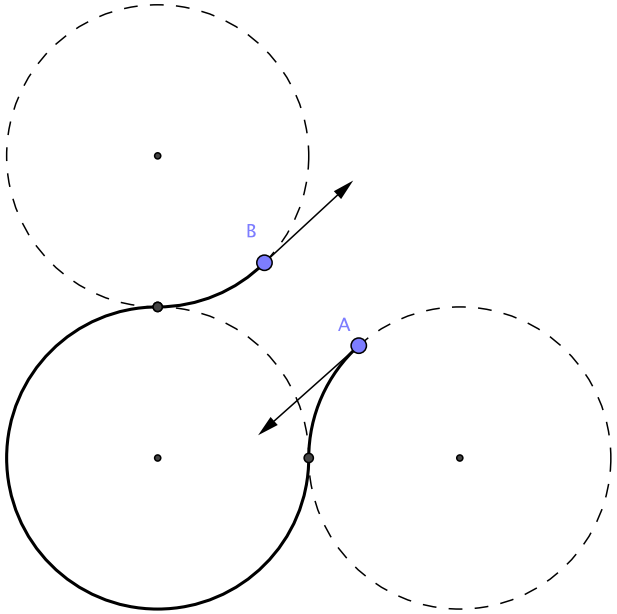
There are 6 possible optimal combinations:

RSR, LSR, RSL, LSR, RLR, LRL

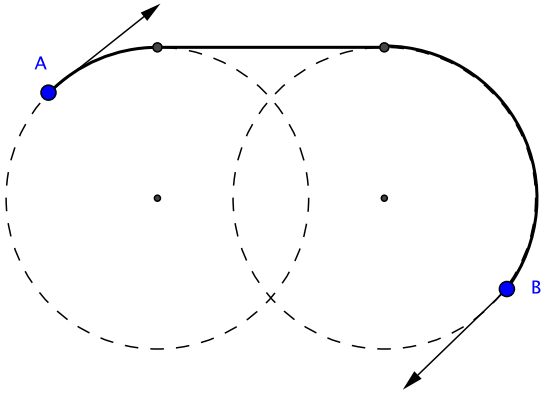
They are highly useful for fixed-wing drones or robots that cannot drive backward.

Hybrid A* - Dubins curves

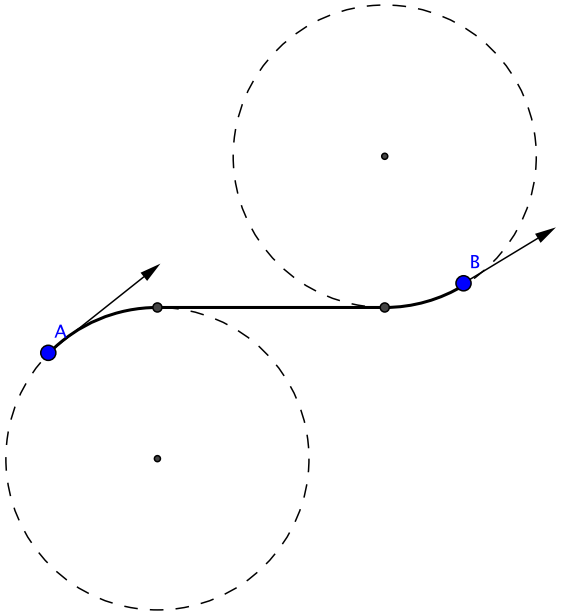
RSL



RSR



LRL



https://en.wikipedia.org/wiki/Dubins_path

Hybrid A* - Reeds-Shepp Curves

If a robot can drive in **reverse**, the shortest path changes.

Reeds-Shepp curves extend Dubins curves by allowing the vehicle to shift into reverse (cusps).

This results in 48 possible optimal path combinations (e.g., Forward Right, Reverse Left, Forward Straight).

What is a real-world driving maneuver that uses a Reeds-Shepp curve?

Parallel parking!

Hybrid A* - Reeds-Shepp Curves

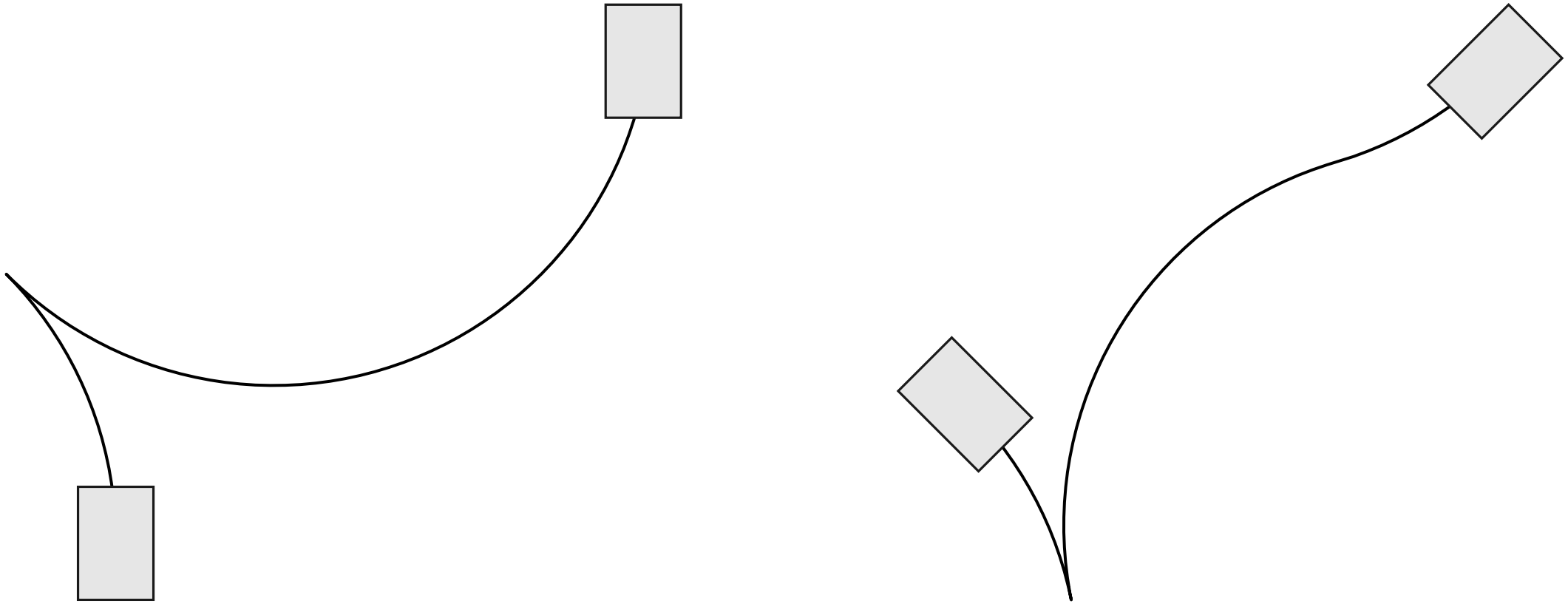
To speed up the search, Hybrid A* uses an "analytic expansion.“

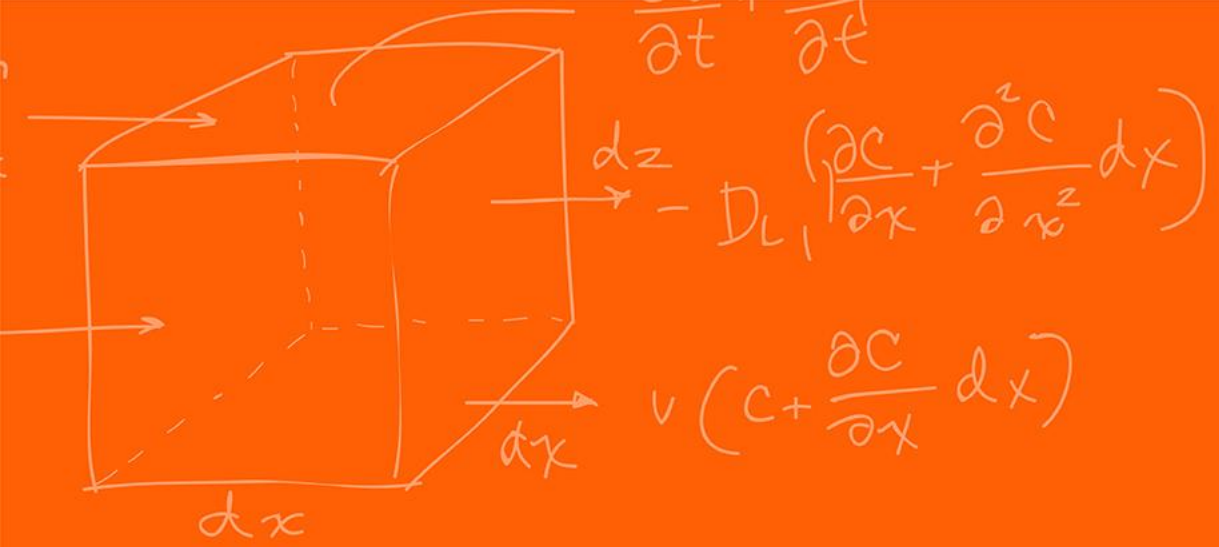
Every few nodes, the algorithm attempts to connect the current state directly to the goal using a Reeds-Shepp curve.

If that continuous curve doesn't hit any obstacles, the search is finished instantly!

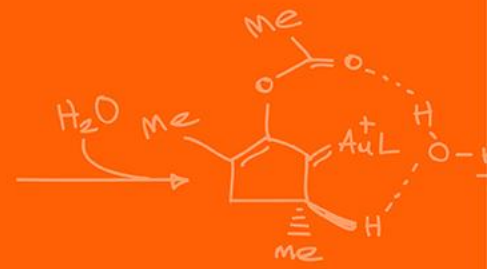
This skips thousands of discrete node expansions near the end of the path.

Hybrid A* - Reeds-Shepp Curves



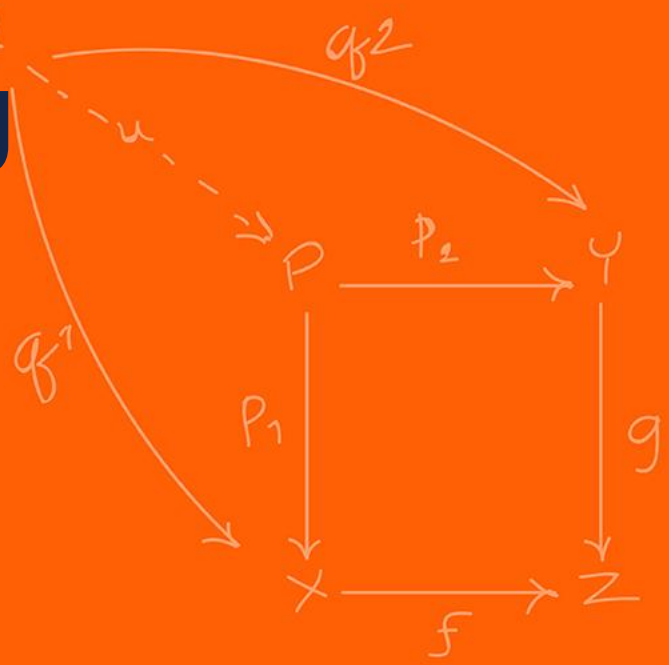








Trajectory Planning



Trajectory Planning

Hybrid A* gives us a smooth, drivable path (geometry).

However, a path is purely spatial. It lacks the dimension of time. A trajectory defines exactly when the robot should be at what state.

Trajectory Planning maps time t to $\big(x(t), y(t), \theta(t)\big)$.

To plan trajectories, we must expand our state space again. The state vector becomes $(x, y, \theta, v, \omega)$.

We now care about the robot's current speed and its rotational speed.

This is required because we must respect the physical limits of our DC motors.

Trajectory Planning

- Every motor has a max RPM, meaning our robot has a $v_{\max}$ and $\omega_{\max}$.

If a path commands a tight turn at high speed, the inner wheel might be commanded to spin faster than physically possible.

The trajectory planner analyzes the curvature of the path.

It slows down the target velocity v_r before entering a sharp turn.

We cannot change velocity instantly; motors have limited torque (acceleration limit).
We also don't want to change acceleration instantly (jerk limit).

In passenger vehicles, why is minimizing "jerk" so important?

Trajectory Planning

- Instead of discretizing time, we represent the continuous trajectory mathematically.

We use piecewise polynomials (splines) to describe the position over time. For example, a cubic polynomial:

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3$$

Combining multiple polynomials / curve fitting functions between multiple waypoints is called a spline.

If you have 3 waypoints, how many polynomials would you need to derive?

Trajectory Planning

- Polynomials are easy to differentiate.

If position is

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3$$

Velocity is

$$\dot{x}(t) = c_1 + 2c_2 t + 3c_3 t^2$$

Acceleration is

$$\ddot{x}(t) = 2c_2 + 6c_3 t$$

By solving for the coefficients $(c_0 \ldots c_3)$, we solve for the entire kinematic profile!

To solve for the polynomial coefficients, we need boundary conditions.

- **Start condition**: The robot's current position and current velocity.
- **End condition**: The target waypoint position and desired arrival velocity.

4 constraints allow us to perfectly solve a 3rd-order (cubic) polynomial.

$$x(0) = c_0 = x_0, \qquad \dot{x}(0) = c_1 = v_0$$
$$x(\mathrm{T}) = c_0 + c_1\mathrm{T} + c_2\mathrm{T}^2 + c_3\mathrm{T}^3, \qquad \dot{x}(\mathrm{T}) = c_1 + 2c_2\mathrm{T} + 3c_3\mathrm{T}^2$$

Substituting c_0 and c_1

$$c_2 = \frac{3(x_f - x_0) - (2v_0 + v_f)T}{T^2}, \qquad c_3 = \frac{2(x_0 - x_f) + (v_0 + v_f)T}{T^3}$$

Trajectory Planning

- We often use a polynomial of order $n = 5$ to ensure that the motor doesn't experience "infinite" jerk at the start and end.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 1 & T & T^2 & T^3 & T^4 & T^5 \\ 0 & 1 & 2T & 3T^2 & 4T^3 & 5T^4 \\ 0 & 0 & 2 & 6T & 12T^2 & 20T^3 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \end{bmatrix} = \begin{bmatrix} x_0 \\ v_0 \\ a_0 \\ x_f \\ v_f \\ a_f \end{bmatrix}$$

By inverting A (which is non-singular for $T > 0$), you find the unique coefficients that define the smoothest possible path for those specific requirements.

This can further be generalized to any order polynomial,

For a polynomial $x(t) = \sum_{i=0}^{n} c_i t^i$, we can represent the relationship between coefficients and boundary conditions as a linear system:

$$\mathbf{b} = \mathbf{A}\mathbf{c}$$

Where:

- $\mathbf{c} = [c_0, c_1, \ldots, c_n]^T$ is the vector of unknown coefficients.
- $\mathbf{b}$ is the vector of boundary conditions (e.g., $[x_0, v_0, a_0, x_f, v_f, a_f]^T$).
- $\mathbf{A}$ is the constraint matrix derived from the power rule.

Each row of $\mathbf{A}$ corresponds to a specific derivative k at a specific time t. The entry for the j-th column (corresponding to c_j) in that row is:

$$A_{row,j} = \frac{d^k}{dt^k}\left(t^j\right) = \frac{j!}{(j-k)!}t^{j-k}$$

Trajectory Planning

- Often, we don't just "solve" for a curve; we optimize it.

We can define a Cost Function J, usually aiming to minimize total time or minimize jerk.

$$\min J = \int_0^T \left|\left|\dddot{x}(t)\right|\right|^2 dt$$

This is framed as a Quadratic Programming (QP) problem, which microcontrollers can solve in milliseconds.

Why might we want to do the Polynomial trajectory generation instead of the QP optimization?

Trajectory Planning

A state-of-the-art approach for local trajectory planning is **Time-Elastic Bands (TEB)**.

It treats the path like a rubber band stretched between the start and goal.

Obstacles act as repulsive forces pushing the band away.

Kinematic limits (max velocity, minimum turning radius) act as internal tension forces keeping the band physically viable.

Trajectory Planning

Now that we have a dynamically feasible trajectory $\left(x_r(t), y_r(t), \theta_r(t)\right)$ and reference velocities (v_r, ω_r)

We can finally feed this into our continuous feedback controller!

Because the trajectory respects the robot's physical limits, the controller will not saturate.

From this we can thus ensure that we follow the constraints of the robot!

Trajectory Planning

- Discrete (x, y) planning ignores physical reality, leading to control instability.

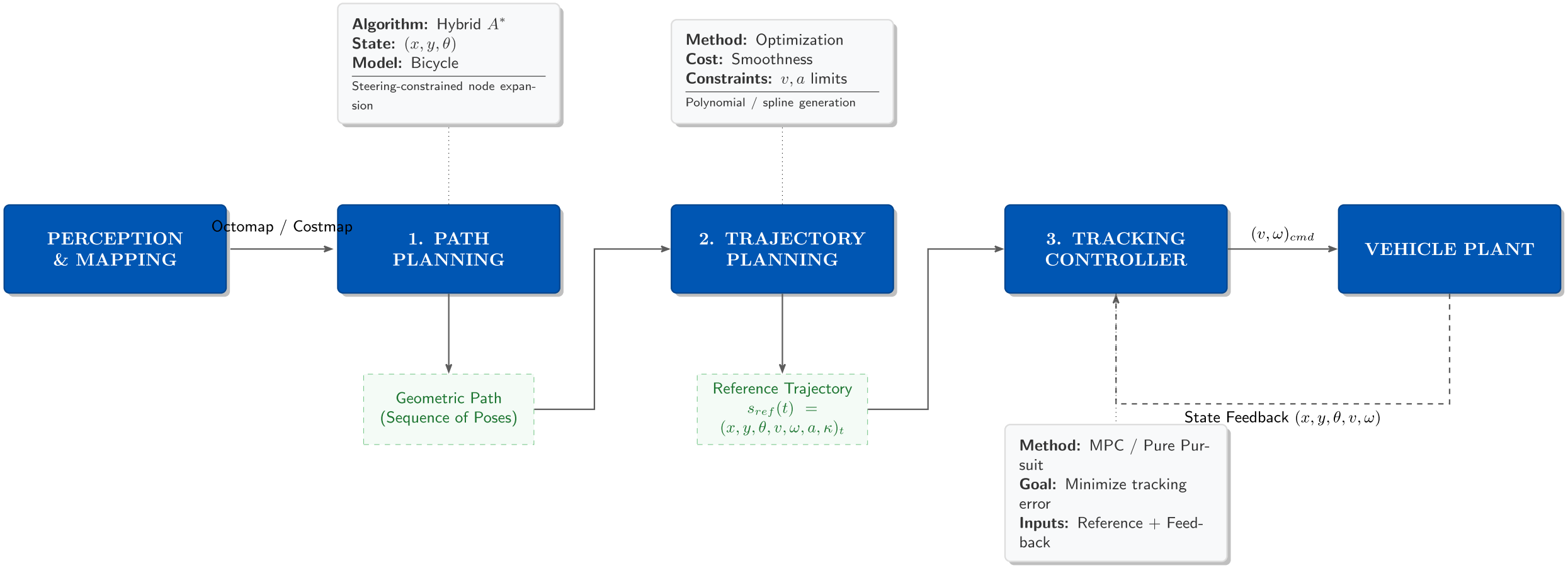
Hybrid A* solves this by planning with kinematic curves in 3D (x, y, θ) space.

Trajectories add the dimension of time and velocity.

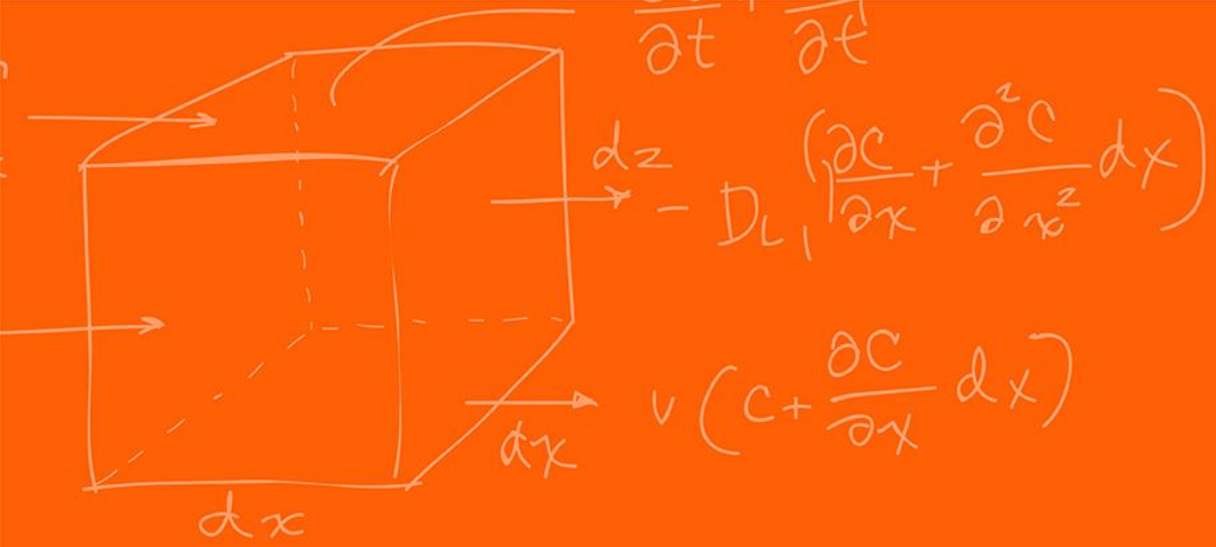
Polynomial optimization allows for smooth acceleration and jerk-free movement.

The controller makes sure that we follow the velocities desired.

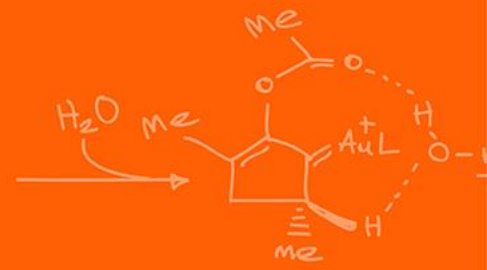
Trajectory Planning



Questions?









The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

